Deep Neural Networks Report Assignment 2.

1 How long did it take for the network to learn?
With basic hardware (CPU), the first basic convolution network took around 15-20 minutes for a full batch of 8 epochs. However with additional GPU processing and a pre-trained application such as VGG16, the time was dramatically reduced to a matter of minutes.

2 Is the data balanced? What is the distribution? If not, what can I do?
The distribution of the chest x-ray images is highly imbalanced. The number of abnormal (bacterial and viral pneumonia) images is much greater than the number of normal images. Ignoring this issue leads the model to heavily bias towards the majority class. This issue was overcome by automatically calculating the class weights during initialisation and feeding them to model.fit() via class_weight. This drives the optimiser to more heavily penalise the minority normal class.

3 Is the model overfitted? Why or why not? If yes, how can I solve this, see GlobalAveragePooling2D?
The original neural network was very prone to overfitting. This happens when the network effectively overfits to the training data by memorising nuances of the training examples rather than extracting generic features and so gets great accuracy on the training set but poor accuracy on the validation set. This problem is partially due to the blind Flatten() layer which feeds a large number of raw parameters into the fully connected layer. Replacing Flatten() with GlobalAveragePooling2D has the effect of averaging the spatial feature tensor map. The network then has the parameters reduced from the hundreds of thousands to just the thousands which significantly restrains the network from being able to memorise any arbitrary patterns of noise and the overfitting problem is thus remedied.

4 Can you do any data augmentation?
Yes. A set of sequential Keras layers was added immediately after the input which uses RandomRotation, RandomZoom and RandomFlip. Using augmentations warps the specific training examples slightly each epoch and forces the convolution filters to identify the main structural characteristics of the diagnosis as opposed to spatial habits preventing short term memorisation and boosting the validation accuracy.

5 What is in the network? Is there anything I can change? Maybe have a look at the Keras tuner.
The initial network used a sequence of Conv2D layers to spot edges and shapes immediately followed by MaxPooling2D for noise reduction and down sampling. This was then followed by a Dense stack. These were replaced by removing the native kernels and mapping the first feature extraction to a deep pre-trained architecture designed and optimised. A library such

as Keras Tuner actually searches through a variety of depths, filter sizes and activation types but it is much better to opt for proven layer elements.

6 Can I use pre-trained model transfer learning?

Transfer learning is strongly recommended for classification of general features on constrained data sets. The new Python implementation loads a pre-trained structural configuration (VGG16) as the top-layer kernel set. The network has organically evolved all the visual primitives it could in millions of hours of viewing common arrangements, which of course translate effortlessly into the constrained field of X-rays because all visual features such as ridges and curves are preserved across images all over the world regardless of medical circumstances.

7 How do you calculate the class by class precision, recall and F1? What do they mean and what's best here?

The last script generates precision and recall using Scikit Learn tools matrix generation. Precision measures the mathematical pureness that a model prediction for a positive event is actually true. Recall is the measure of how well a model sweeps across an environment of possible inputs and catches all potential targets without false negatives. F1 is a harmonic normalisation point which translates the success of the first two factors into a single factor. For pneumonia detection, missing a deadly patient is far worse than the inconvenience of a second medical test for a well patient thought to have pneumonia. Hence Recall being the best structural diagnostic factor.

8 How can you improve the model to detect the sick?

Increasing true sick cases heavily relates to increasing the general recall. By manually increasing the proportional weight of the sick classes, the model's parameter generation cycles weight the exponential importance of capturing sick properties significantly over capturing healthy properties thus dramatically reducing false negatives directly on the patients. Or manually lowering the binary prediction sigmoid function value from the default 0.5 decision point to a conservative 0.3 decision point linearly downwards will allow the model to immediately trigger sickness alerts for much lower associative presence values detected generally resulting in high recall outcomes.

9 Can I see what the CNN is looking at? Can I see this using tf-explain?

Yes. To achieve this a GradCAM element was fully created using visual gradient descent maps natively generated within tf-explain. To run the gradient visual models effectively maps the internal numeric matrix activation biases (weights) generated by the convolutional network to essentially blend mathematical heatmap outputs globally onto normal test output slices. The current Python pipeline embodies this effectively to unambiguously reveal what particular parts of the rib cage structure were effectively used in diagnostic processing calculations in general automatically displaying the true visual interpretation of this decision mapping.

10 Any other things you discover to optimise the model?

Explicitly decreasing learning rates over later epochs via callbacks ensures the learning algorithm gradients do not excessively flop around essentially letting the processing descent limits gradually decay gently directly onto final mathematical optimum points thus ensuring overall system performance processing normalisation across arbitrary epoch sequences automatically. Regularisation logic variables via dynamic dropout sequences also drives structural data pathways along variable pathing routes avoiding specific strong dependency processing errors.